

テクノロジー - ソフトウェア開発管理技術

1. 「スクラムチーム (Scrum Team)」 - 「プロダクトオーナー (PO / 产品负责人)」

役割	イベント	作成物
プロダクトオーナー	スプリント	プロダクトバックログ
開発チーム	スプリント計画	スプリントバックログ
スクラムマスター	デイリー・スクラム	増分 (インクリメント)
	スプリントレビュー	
	スプリントレトロスペクティブ	

図 スクラムの構成要素

プロダクトオーナー (PO) の核心定位：

- 他是 “价值的守门人” 。
- 他的唯一职责是：确保团队做出来的东西（Product）是最有商业价值的。

PO 的核心权限：

- 他拥有 「プロダクトバックログ (Product Backlog / 产品待办列表)」 的最高指挥权。他负责决定做什么、不做什么，以及任务的优先级（顺序）。

我们用 “价值守护者” 的视角来审视这几个选项，你会发现其他选项全是来 “乱入” 的：

- ア：生み出されるプロダクトの価値を最大化するために、プロダクトバックログのアイテムを作成し、並び替える。
 - ☒ 正确答案：完美命中！PO 的使命就是 “最大化价值”，而实现这一点的手段就是控制 “产品待办列表” 的增删和优先级排序。
- イ：完成の定義を忠実に守ることにより品質を作りこみ、利用可能なインクリメントを完成させる。
 - ☒ 错误：这是 「開発チーム (Development Team)」 的职责。开发团队负责交付高质量的 “增量 (Increment)” 。
- ウ：スプリントの計画を作成する。
 - ☒ 错误：这是整个 「スクラムチーム (Scrum Team)」 在 “冲刺规划会议” 上的共同协作结果，而非 PO 独自承担的任务。
- エ：チームのリーダーとして、自己管理型で機能横断型のチームのメンバーをコーチする。

- **✗ 错误**：这是「スクラムマスター (Scrum Master)」的职责。他是团队的教练，负责扫清障碍、促进流程。

把「PO、开发团队、Scrum Master」脑补成【老王餐厅的‘铁三角’】：

 老王餐厅的分工现场：

- **PO (产品负责人)**：是“金牌推销员”。他站在门口看客人最想要什么菜，然后写下菜单（产品待办列表），告诉厨师：“今天先做这个，再做那个，这个最有利润！”（**决定价值与顺序**）。
- **开发团队**：是“厨师团队”。他们负责把菜做出来，而且必须严格按照卫生标准（**完成的定义**）做。
- **Scrum Master**：是“厨房经理”。他专门给厨师磨刀、修油烟机、协调各环节，确保厨师做饭不卡壳。

记住：PO = “**价值指挥官，只管菜单顺序和产品赚钱与否**”

口诀唤醒记忆：

价值最大是使命，PO 手握名单行。待办列表增删改，排好顺序听我令。开发负责出成品，教练 SM 扫瓶颈！

 “敏捷开发角色”考场的“机械化”秒杀公式：

在考场上看到问「プロダクトオーナー」：

- 关键词扫描**：寻找「**価値最大化 (价值最大化)**」、「**プロダクトバックログ (产品待办列表)**」、「**優先順位 (优先级)**」。只要出现这些词，直接闭眼选它。
- 绝对排雷（见到就划掉）**：
 - 凡是说“辅导成员”、“流程优化”的 → **通通划掉，这是 Scrum Master (SM)!**
 - 凡是说“完成工作”、“高质量交付”的 → **划掉，这是开发团队 (Development Team)!**

2. 「構成管理 (Configuration Management)」

核心定义：构成管理不是简单的“归档”，它是对软件开发过程中所有“零件”（源代码、设计书、说明书等）的**全生命周期溯源**。

核心目的：解决“开发混乱”问题，确保当有人问“现在线上跑的是哪个版本？”或“哪个修改导致了 Bug？”时，能迅速通过“管理台账”查明真相。

核心三要素：

- **识别 (Identification)**：给每个零件贴标签（版本号、ID）。
- **记录 (Recording)**：记录谁改了什么、什么时候改的（变更履历）。
- **追踪 (Tracing)**：保证最新版本和发布状态清晰可见。

根据“构成管理”的定义，考试中常见的**干扰陷阱**和**正确逻辑**如下：

- **陷阱 1：构成管理 = 备份？**

- **✗ 错误**：备份只是保存，而构成管理是“控制”和“追踪”。单纯备份解决不了版本冲突和修改溯源问题。

○ **陷阱 2：构成管理 = 仅仅管理代码？**

- **✗ 错误**：这是新手最容易掉的坑。设计书、手册、库文件 (Library) 统统属于“构成品目”。漏掉任何一个文档，都会导致版本不匹配的灾难。

○ **核心判断标准 (正确逻辑)：**

- 凡是提到「**变更内容 (变更内容)**」、「**最新のバージョン (最新版本)**」、「**管理台帳 (管理台账)**」且强调「**構成要素 (构成要素/零部件)**」的，就是构成管理。

把「**構成管理**」脑补成【老王餐厅的‘超级账本’】：

📖 老王的‘零件身份证账本’：

- 老王餐厅做菜，不是只有菜，还有菜谱（设计书）、秘制酱料（库文件）、操作说明（手册）。
- 老王给每一个东西都发了身份证（版本号）。
- 厨师改了一次配方（变更内容），必须在“超级账本”（管理台账）上写清楚：“几月几日，张三改了酱料，V1.0 变成了 V1.1”。
- 口诀：零件改了要登记，版本身份证别丢弃。谁改的，改了啥，账本一查全都有！

🎯 “构成管理”考场的“机械化”秒杀公式：

a. **关键词扫描**：在选项中寻找以下三个组合：

- “部品 (零部件/构成要素)” + “变更内容 (变更内容)” + “バージョン番号 (版本号)”。
- 只要这三个要素凑齐，这就是正确选项。

b. **避坑指南**：

- 如果选项里写着“只针对源代码” → **叉掉！** (文档也算)。
- 如果选项里写着“为了性能优化” → **叉掉！** (这是性能测试的目的，不是构成管理的目的)。

3. 「リバースエンジニアリング (Reverse Engineering / 逆向工程)」

什么是 リバースエンジニアリング (Reverse Engineering)？

- **核心定义**：它是“由果推因”的过程。当你拿到一个成型的、甚至没有文档的“成品软件”时，通过技术手段将其“拆解”，还原出它的设计蓝图、算法结构和仕様书。

核心逻辑：

- **正向工程**：从蓝图到产品。
- **逆向工程 (Reverse)**：从产品拆回蓝图。

我们用“侦探还原法”来扫清陷阱：

- **ア：CASEツールなどを用いて、設計情報からソースコードを自動生成する**
 - **✗ 错误：**这是「順方向エンジニアリング (Forward Engineering)」。根据设计蓝图做产品，这是标准的正向开发流程。
- **イ：外部から見たときの振る舞いを変えずに、ソフトウェアの内部構造を変える。**
 - **✗ 错误：**这是「リファクタリング (Refactoring / 重构)」。为了代码更干净，在不改变功能的前提下修改内部结构。
- **ウ：既存のソフトウェアを解析し、その仕様や構造を明らかにする。**
 - **✓ 正确答案：**完美命中！解析成品软件，反向推导出仕様 (Specifications) 和构造 (Structure)，正是逆向工程的定义。
- **エ：既存のソフトウェアを分析し理解した上で、ソフトウェア全体を新しく構築し直す。**
 - **✗ 错误：**这是「リエンジニアリング (Re-engineering / 再工程)」。不仅要分析，还要在这个基础上重新构建（重写）。

“逆向工程” 考场的 “机械化” 秒杀公式：

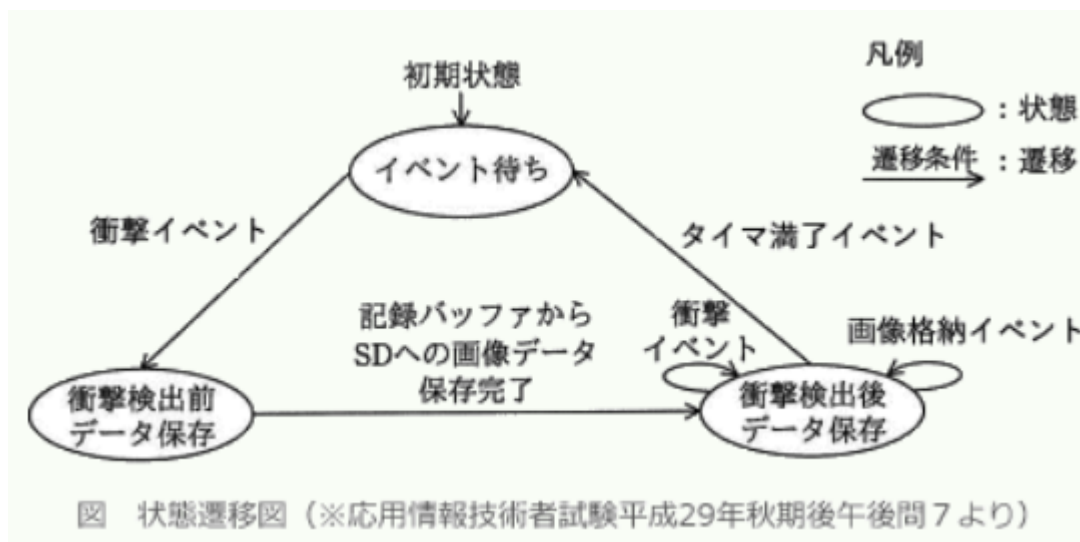
在考场上看到问「リバースエンジニアリング」：

- a. **关键词扫描：**寻找「仕様 (规格)」、「構造 (构造)」、「解析 (解析)」、「明らかにする (明确化)」。
- b. **视觉识别法：**
 - 看到“自动生成代码” → **叉掉** (这是正向)。
 - 看到“重构/保持行为修改结构” → **叉掉** (这是重构)。
 - 看到“重新构建/重做” → **叉掉** (这是再工程)。
- c. **核心精髓：**逆向就是“为了读懂而拆解”，只拆解还原，不涉及重写。

4. 「状態遷移図 (State Transition Diagram / 状态迁移图)」

什么是 状態遷移図 (State Transition Diagram)?

- **核心定义：**它专门用于描述一个系统在“不同的状态”之间是如何根据“事件/输入”进行跳转的。
- **核心逻辑：**如果一个系统有着清晰的“生命周期”或者“开关状态”，且状态的数量有限、变化规则明确，那就是状态迁移图的“主场”。



核心考点——“选择标准”：

- 适用对象：有顺序、有状态、受外部触发影响的控制系统或交互系统。

我们用“状态扫描仪”来甄别哪个系统最需要“状态迁移”：

- ア：月末及び決算時の棚卸資産を集計処理する在庫棚卸システム
 - ✗ 错误：这是典型的「バッチ処理 (批处理)」系统。它看重的是数据的计算和汇总，而不是“状态变化”，通常用流程图 (Flowchart) 或数据流图 (DFD) 即可。
- イ：システム資源の稼働状態を計測し、レポートとして出力するシステム資源稼働状態計測システム
 - ✗ 错误：这是典型的「監視・集計 (监控/统计)」系统。它是一个持续运行的监测过程，不存在复杂的“状态跳转”逻辑。
- ウ：水道の検針データから料金を計算する水道料金計算システム
 - ✗ 错误：这是典型的「事務処理 (事务处理)」系统。本质是数据输入 -> 计算 -> 输出，逻辑比较单一，不属于状态驱动。
- エ：設置したセンサーの情報から、温室内の環境を最適に保つ温室制御システム
 - ✓ 正确答案：完美命中！这类控制系统有非常明确的状态，例如：“温度低时（触发条件）” -> 进入“加温状态”；“温度适中时（触发条件）” -> 进入“停止/待机状态”。这种根据传感器输入在不同动作状态间切换的逻辑，最适合用 **状态迁移图** 来描述。

把「状态迁移图」脑补成【老王餐厅的‘电灯开关守则’】：

💡 老王的‘灯光指挥官’：

- 老王餐厅的灯有两种状态：【亮】和【暗】。
- 如果现在是【暗】，按一下开关（触发条件），它就迁移到【亮】。
- 如果现在是【亮】，按一下开关（触发条件），它就迁移到【暗】。
- 判断秘籍：凡是那种像“灯泡”一样，能在“待机”、“运行”、“报警”、“停止”之间跳来跳去的系统，就是状态迁移图的菜！

□ 口诀：事务处理用流程，状态切换用迁移。凡是“触发后跳变”的，必选状态迁移图！

🎯 “状态迁移图”考场的“机械化”秒杀公式：

在考场上看到问「状態遷移図を用いることが最も適切なシステム」：

- a. 关键词扫描：寻找「制御 (控制)」、「状態 (状态)」、「センサー (传感器)」、「イベント/トリガー (事件/触发)」。只要看到这些，直接选它。
- b. 避坑指南：
 - 凡是看到“计算”、“汇总”、“报告输出”、“数据处理”等关键词的 → 通通划掉！这类系统更偏向于数据流图 (DFD)。
- c. 最终判定：如果一个系统在运行时，你能清晰画出它从 A 状态切换到 B 状态的条件，那它就是状态迁移图的目标。

5. 「マッシュアップ (Mashup / 混搭)」

什么是 マッシュアップ (Mashup)?

- 核心定义：它不是从零写代码，而是“拼接”。它通过调用不同企业公开的 **API (Application Programming Interface)**，将别人的服务（比如地图、天气、新闻）“混搭”在一起，生成一个全新的、具有独特价值的 Web 应用。

核心考点——“拼接而非重写”：

- 关键点：不需要理解底层的复杂实现，只需要利用现成的 API 进行“组装”。

在 FE 考试中，关于“マッシュアップ”的题目通常会设置以下陷阱：

- 陷阱 1：与“系统集成 (System Integration)”混淆
 - 辨析：系统集成通常是企业内部的深度架构融合；而 Mashup 更侧重于“Web 服务间的 API 拼接”，侧重于快速开发。
- 陷阱 2：与“封装/插件”混淆
 - 辨析：Mashup 是服务级别的拼接（例如：地图+酒店信息），而不是代码库的调用。
- 核心判断逻辑：
 - 只要选项中提到「複数のWebサービス (多个Web服务)」、「APIの組み合わせ (API组合)」、「短期間で開発 (短时间开发)」，这必然就是 Mashup 的正确描述。

🥗 老王的‘混搭拼盘’：

- 老王不想自己辛苦去种菜（不需要自己开发后端）。
- 他去邻居菜市场买点现成的生菜，去隔壁摊位买点现成的熟肉（调用公开的 API）。
- 老王把这些现成的食材直接拼成了一个全新的“超级招牌沙拉”（全新的 Web 服务）。
- 这个“拼盘”的过程，就叫 Mashup！

记住：Mashup = “别家现成菜，拿来拼拼看，做出新招牌，省力又快办”

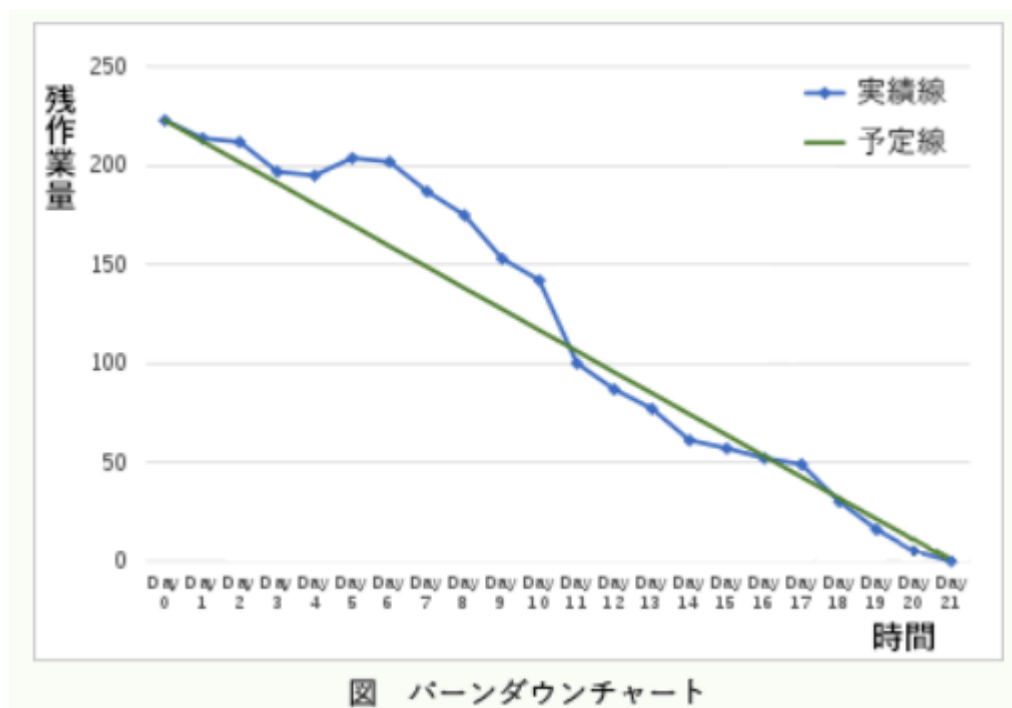
口诀唤醒记忆：

Mashup 叫混搭，API 组合是法宝。借用他人好服务，开发时间省不少。深层逻辑不用管，拼成新品价值高！

🎯 “混搭 (Mashup)” 考场的“机械化”秒杀公式：

在考场上看到问「マッシュアップ」：

- 关键词扫描：**寻找「Webサービス (Web服务)」、「組み合わせ (组合)」、「API」、「短期間 (短时间)」。只要这几个词同时出现，直接选它！
 - 绝对排雷 (见到直接 Pass)：**
 - 凡是提到“内部系统重构”、“底层源代码修改”、“系统性能优化”的 → **通通划掉！**
Mashup 的核心是“拼接现有能力”，而不是“底层修改”。
 - 视觉识别：**Mashup 通常与地图 API (Google Maps) 结合展示数据，这是该类题目最常见的考点图景。
6. 「バーンダウンチャート (Burndown Chart / 燃尽图)」



什么是 バーンダウンチャート？

- 核心定义：它是敏捷开发（アジャイル開発）中，用来展示“项目还有多少活儿没干完”的可视化工具。

核心考点——“坐标系与曲线含义”：

- 纵轴 (Y轴)：残作业量 (Remaining Work)。
- 横轴 (X轴)：时间 (Time/Days)。
- 予定線 (绿色直线)：理想状态下的进度。
- 実績線 (蓝色折线)：真实的进度。

判定逻辑：

- 线在上方：干得太慢，**延期了**。
- 线在下方：干得飞快，**超前了**。

虽然当前图片主要是解说，但我们需要从题目角度透视它的“考场逻辑”：

◦ **陷阱 1：反向理解**

- 很多同学看到蓝色折线往上翘（比如图中的第 5 到 6 天），误以为是“进度加快”。
- **真相**：在燃尽图中，“向上翘”意味着**残余作业量增加了**（可能是加了新需求或发现漏估了），这是**严重的进度退后信号**。

◦ **陷阱 2：直线代表什么**

- 绿色直线是“**完美假设**”。现实中，进度线总是波动的，只要整体趋势在绿色直线附近，就说明进度是受控的。



老王的‘剩菜消灭记’：

- 老王今天准备了 250 份食材（纵轴），要在 Day 21 前全部卖完。
- 绿色线是他心里想的：“每天卖多少，剩下的就成一条直线”。
- 蓝色线是实际情况：“哎呀，今天没卖出去（线往上翘了）”、“哇，今天爆单了（线垂直往下掉）”。

记住：燃尽图，就是“剩饭记录本”。线越低，代表剩饭越少，工作干得越爽！

口诀唤醒记忆：

横轴时间纵轴残，绿色直线是理想。蓝色折线若在上，加班加点把进度抢。线若向下快速掉，敏捷开发真有效！



“燃尽图”考场的“机械化”秒杀公式：

a. 视觉直觉法：

- 看到题目问：“进度比计划慢吗？” → **看蓝线是否在绿线“上方”！**
- 看到题目问：“怎么做才能让它变好？” → **找“调整范围”或“增加资源”的选项！**

b. 避坑指南：

- 蓝线翘起不代表“做完了”，代表“活儿变多了/进度倒退了”。考试中如果问这代表什么，一定要选“进度延期或新需求追加”。

7. 数据库建模与逆向工程-リバースエンジニアリング

什么是“从数据库定义信息还原设计书”？

- **核心定义**：这是典型的「**由果推因**」的操作。因为数据库已经运行了（果），我们根据它现有的定义（表结构、字段、关系），反向画出 E-R 图（因/设计蓝图）。

リバースエンジニアリング (Reverse Engineering / 逆向工程)

- 题目描述的是“从正在运行的数据库定义中生成 E-R 图”，这正是将“代码/系统”倒推回“设计文档”的过程，完全符合逆向工程的定义。

核心考点——“工程方向”：

- **顺向 (Forward)**：从设计图 → 数据库代码。
- **逆向 (Reverse)**：从数据库代码 → 设计图 (E-R 图)。

老王的‘逆向复盘记’：

- 老王餐厅的菜（数据库）已经做好了。
- 老王想复盘一下这道菜是怎么做出来的（生成 E-R 图设计书）。
- 他盯着做好的菜，反推它的配方和结构，这就像是“把做好的实物拆回设计图”。

记住：只要是把**实物/运行代码**拆解还原成**文档/蓝图**，它就是逆向工程！

口诀唤醒记忆：

蓝图变代码，这是顺方向。代码变蓝图，逆向是真相。数据还原 E-R 图，逆向工程名声响！

“工程方向”考场的“机械化”秒杀公式：

a. 关键词扫描：

- 看到“设计书 -> 代码/数据库” → 锁定 **Forward (顺向)**。
- 看到“数据库/系统 -> 设计书/E-R图” → 锁定 **Reverse (逆向)**。

b. 绝对排雷 (见到就叉掉)：

- 凡是出现“社会工程学”的 → **通通划掉**，它是骗子行为，跟软件开发没关系。
- 凡是出现“并行工程”的 → **划掉**，那是时间管理方法。

8. 「ソフトウェアのライセンス (Software Licensing)」--这道题考察的是你对“使用许诺 (Usage License)”本质的理解，切忌被各种复杂的专利权、产品形态干扰。

什么是软件使用许诺 (使用許諾 - License)?

- **核心定义**：你是软件的著作权人（或拥有者），你赋予别人“在一定条件下使用该软件”的权利。这是一种“赋予权限的行为”。

核心逻辑：

- 软件的著作权和专利权是独立的。
- 即使没有专利保护，只要你是版权人，你就有权出租或授权别人使用你的“作品”（软件）。

特許で保護された技術を使っていないソフトウェアであっても、使用許諾することは可能である。

✅ **正确答案**：完美命中！软件的核心权利是“**著作权 (Copyright)**”。只要这代码是你写的，即便它里面没有任何“专利技术”，你依然是版权人，你依然可以授权别人使用。

老王的‘红烧肉秘籍’：

- 老王写了一本《红烧肉制作手册》（**软件源代码**）。
- 不管这本手册有没有去申请“特殊锅具专利”（**专利权**），这手册是老王写的，版权就是老王的（**著作权**）。
- 老王想卖给隔壁餐厅使用权，他想单独卖书就卖书，想连锅一起卖就一起卖。只要老王同意，别人就能用。

记住：授权靠的是版权，不是专利。只要你是作者，你想怎么授权都行！

口诀唤醒记忆：

授权本质看著作，专利无需捆绑缚。单体售卖无限制，无偿不是 OSS 途。版权在手权我有，合法授权路路通！

 **“软件授权”考场的“机械化”秒杀公式：**

- 核心判断：**软件授权的核心是“**著作权 (著作權)**”，不是“**专利权 (特許權)**”。看到选项把专利权和授权捆绑的，一律视为干扰项。
- 绝对避雷：**
 - 凡是看到“由于 A，所以不能做 B”（且 A、B 逻辑不相关）的描述，通常都是错的。
 - 凡是看到“无条件”、“一定会变为”、“只有...才能...”这种绝对化表述的，在法律题中 90% 是错的。
- 最终逻辑：**只要软件是你的，你就有权授权，无论它是否复杂、是否带专利、是否免费。

9. 「組込みシステムにおける特許ライセンス (嵌入式系统中的专利许可)」

- 这个知识点的核心在于：**硬件和软件在法律上是“分家”的，即便它们在物理上“绑定”在一起。**

硬件与软件的独立性：

- 在法律（专利法）层面上，**硬件专利** 和 **软件专利** 是两个独立的护城河。
- 即便一个嵌入式系统将软硬件封装得天衣无缝，如果你使用了别人的技术，你必须向“硬件专利权人”和“软件专利权人”分别申请实施许可 (License)。

核心考点——“独立授权逻辑”：

- **不能“一码通吃”：**买了一家的硬件许可，不代表你自动获得了配套软件的专利使用权，反之亦然。

针对这道题目描述的逻辑，我们在考场上需要特别警惕以下判断陷阱：

- **干扰逻辑 1：“一体化误区”**
 - **陷阱：**因为硬件和软件结合得非常紧密（密なつながり），所以认为获得其中一方的授权就等于获得了另一方的授权。
 - **真相：**法律不看物理连接。它们是「**別の権利 (独立的权利)**」，这是解题的唯一标准。
- **干扰逻辑 2：“实施权的必要性”**

- **真相：**如果你使用了他人的技术，“**特許の実施権 (专利实施许可)**”是必须获得的。不获得许可即属于侵权。

🛡️ 老王的‘分家授权书’：

- 老王餐厅出了一款“特制煎鱼套餐”，用的是：特制铁板（**硬件**）+ 祖传秘制酱汁（**软件**）。
- 这铁板是老王租李厂长的，酱汁是老王买王师傅的。
- **逻辑点：**就算老王把铁板和酱汁牢牢粘在一起，你也得分别去求李厂长许可你用铁板，求王师傅许可你用酱汁。你不能因为李厂长同意了，就默认王师傅也同意。

记住：**软硬虽一体，专利要分家。一家要一证，否则全犯法！**

口诀唤醒记忆：

软硬嵌入紧相连，法律权利两边看。硬件特许归硬件，软件专利另需办。两家许可皆要请，漏掉一个就维权！

🎯 “专利授权”考场的“机械化”秒杀公式：

- 关键词锁定：**看到「**別の権利 (独立权利)**」必选。看到暗示“获得一方许可即可自动获得另一方”的描述，直接排除。
- 绝对排雷：**
 - 凡是说“因为连接紧密，所以权利合一”的 → **通通划掉！** 法律面前没有“物理连接”这一说。
 - 凡是说“因为买了硬件，所以软件免费用”的 → **划掉！** 这是典型的侵权思维。

10. 「CMMI (Capability Maturity Model Integration)」

CMMI 的核心定位：

- 它是企业的“**能力段位表**”。通过评估，判断一个组织在项目管理和开发上处于哪个“段位”。

核心逻辑——“由乱到治，再到巅峰”：

- CMMI 通过 5 个阶段（Level 1-5）描述了企业流程从“凭运气”到“凭数据”，最终到“自动进化”的演进路径。

核心术语：

- **ゴール (Goal/目标)** 与 **プラクティス (Practice/实践)**：这是达成成熟度的砖瓦。

针对 CMMI 五个段位，考场上极易混淆，我们进行深度拆解：

- **レベル1：初期 (Initial)**
 - **逻辑：**完全靠英雄主义或运气。今天项目成功可能纯属巧合，一旦主力离职，项目就崩。
- **レベル2：管理された (Managed)**

- **逻辑**：开始有了项目管理的基本动作（比如进度表、预算控制）。但只是“单个项目”内做得好，还没上升到组织高度。
- **レベル3：定義された (Defined)**
 - **逻辑**：把 Level 2 的优秀经验变成“**组织标准**”。全公司不论谁做项目，都得用同一套规范，有一致性。
- **レベル4：定量的に管理された (Quantitatively Managed)**
 - **逻辑**：进入“大数据时代”。通过收集历史数据，用「**データに基づき (基于数据)**」的方式进行预测。能说出“如果我们这么改，Bug 率会下降 X%”。
- **レベル5：最適化している (Optimizing)**
 - **逻辑**：进入“自动进化时代”。不只管流程，还主动寻找创新的机会，流程自己会优化，不仅稳，而且一直在变强。

把「CMMI 5 个段位」脑补成【老王餐厅的‘连锁化进化史’】：

 老王的‘五个段位段子’：

- **Level 1 (初期)**：厨师心情好菜就好，心情差就毁了。（**全凭运气**）
- **Level 2 (管理)**：厨师长开始写笔记，把每个项目记录下来。（**有了基本规矩**）
- **Level 3 (定义)**：全连锁店统一菜谱，所有厨师做法一致。（**有了统一标准**）
- **Level 4 (定量)**：用秤量盐，记录数据，预测成本和味道。（**数据驱动预测**）
- **Level 5 (优化)**：厨师根据潮流自动研发新菜单，流程天天优化。（**自动进化**）

记住：**运气 -> 记账 -> 统一 -> 秤量 -> 进化**，这就是 CMMI 的五步走！**

 “CMMI”考场的“机械化”秒杀公式：

a. 关键词锁定：

- 看到“**场当たりの (临时抱佛脚/无序)**” → Level 1。
- 看到“**项目层面**” → Level 2。
- 看到“**组织层面/标准化/一致性**” → Level 3。
- 看到“**定量/数据/预测/统计**” → Level 4。
- 看到“**持续改善/优化**” → Level 5。

b. 绝对避雷：

- 别把 Level 3 和 Level 4 搞混。Level 3 是“大家都按规定办”，Level 4 是“按数据说话办”。

11. 「SOA (Service Oriented Architecture / 面向服务的架构)」

什么是 SOA？

- **核心定义**：它是一种“**化整为零，按需拼搭**”的系统架构思想。将原本铁板一块（Monolithic）的庞大系统，拆解成一个个独立、松耦合的“服务 (Service)”。

核心考点——“解耦与组合”：

- **独立性**：每个服务（如支付、用户管理）都是独立的，互不干涉。
- **标准接口**：各服务通过标准接口（API 等）互相通信，就像乐高积木一样，不管内部怎么变，接口对上就能用。
- **灵活性**：修改支付功能，不会导致整个系统崩溃，甚至可以复用到其他系统中。

在考试中，SOA 的“死敌”选项通常如下：

- **干扰项：“完全依赖特定平台”**
 - **✗ 错误**：SOA 的精髓在于“**中立性**”。它不依赖于特定的开发语言或平台（因为有标准接口），如果选项说它“严重依赖 Windows 平台”，直接叉掉。
- **干扰项：“单一整体化开发”**
 - **✗ 错误**：SOA 坚决反对整体开发，它主张“**独立开发、独立部署**”。
- **正确逻辑 (关键词锁定)**：
 - 凡是提到「**再利用性 (可复用性)**」、「**独立性 (独立性)**」、「**標準化インターフェース (标准化接口)**」、「**柔軟なシステム構築 (灵活构建)**」的选项，就是正确答案。

把「SOA」脑补成【老王餐厅的‘乐高积木厨房’】：

老王的‘乐高厨房守则’：

- 以前老王做饭是一个人从头忙到尾（**整体架构**），一旦切菜的累了，整个餐厅停摆。
- 后来老王引入了 SOA，把厨师分成了：切菜员、炒菜员、洗碗工（**服务/Service**）。
- 他们通过“传菜窗口”沟通（**标准接口**）。
- 如果切菜员想换个型号的刀，根本不影响炒菜的，甚至还能去帮隔壁餐厅切菜（**可复用、低耦合**）。

记住：SOA = “**模块化拼搭，接口一插就搞定，换个功能不用重做全套**”

口诀唤醒记忆：

业务拆分变服务，独立运行逻辑组。接口标准要统一，灵活拼搭最稳固。开发效率翻倍涨，复用方便系统酷！

“SOA”考场的“机械化”秒杀公式：

- 关键词锁定**：在选项中寻找「**サービス (服务)**」、「**再利用 (再利用)**」、「**インターフェース (接口)**」、「**疎結合 (松耦合/低耦合)**」。只要看到这几个词，它就是 SOA。
- 绝对避雷**：
 - 凡是提到“通过直接修改源代码实现功能” → **叉掉！** SOA 讲究的是通过调用接口。

- 凡是提到“紧耦合 (密結合)” → 叉掉！这是 SOA 的反义词。

12. 「XP (Extreme Programming / 极限编程)」

- XP 的核心哲学是“把敏捷开发中的优秀实践做到极致”，我们要抓住它那些“简单粗暴但高效”的关键词。

什么是 XP (Extreme Programming)?

- **核心定义**：它是一种注重“代码质量”和“沟通协作”的软件开发方法，提倡短周期、高反馈。

核心考点——“XP 五大价值观与实践”：

- **CI (Continuous Integration / 持续集成)**：这是 XP 的心脏。它要求频繁地把代码合并到主干并进行自动化测试，确保“集成”不痛。
- **TDD (Test Driven Development / 测试驱动开发)**：先写测试，再写代码。
- **リファクタリング (Refactoring)**：随时随地优化代码。（在维持软件「外部功能行为 (外部から見たときの振る舞い)」完全不变的条件下，通过调整「内部结构 (内部構造)」，以实现提升代码可读性、降低维护成本的过程。）

コードの結合とテストを継続的に繰り返す。

- ☒ **正确答案**：这正是 XP 中「継続的インテグレーション (Continuous Integration / 持续集成)」的核心定义。通过不断地合并、测试，把问题消灭在萌芽状态。

共同のプラクティス	開発のプラクティス
反復 共通の言語 オープンな作業空間 回顧	テスト駆動開発 ペアプログラミング リファクタリング ソースコードの共同所有 継続的インテグレーション YAGNI
管理者のプラクティス	顧客のプラクティス
責任の受入れ 援護 四半期毎の見直し ミラー 最適なペース	ストーリーの作成 リリース計画 受入れテスト 短期リリース

図 XPのプラクティス

把「XP 实践」脑补成【老王餐厅的‘超速流水线’】：

老王’s ‘超速流水线’ 守则：

- 老王餐厅为了追求速度 (**Extreme**)，定下了三条规矩：
 - a. **代码总合 (CI)**：就像厨师每切完一个配菜，立马传到传送带上检查，不积压 (**持续集成**)。
 - b. **测字驱动 (TDD)**：没写菜谱前，先写“好吃标准”。必须先过标准，才能做菜。
 - c. **随时整形 (Refactoring)**：觉得菜盘摆得不好看，随时调整，不用等换班。

记住：XP = “**速度快、节奏紧、先测后写、随时修整**”

口诀唤醒记忆：

XP 实践追求快，持续集成心不坏。测试驱动是先行，重构随时不用待。顾客验收分工明，质量效率并排在！

🎯 “XP 开发实践”考场的“机械化”秒杀公式：

- a. **关键词扫描**：
 - 看到「**継続的インテグレーション (持续集成)**」或「**結合とテスト (结合与测试)**」，直接选那个提倡“重复、快速、反复”的选项。
- b. **绝对排雷 (见到直接 Pass)**：
 - 看到“不重构” → **叉掉！** XP 视代码质量如生命，绝不可能不重构。
 - 看到“顺序颠倒 (先写代码后写测试)” → **叉掉！** 这是 TDD 的死敌。
 - 看到“顾客负责单元测试” → **叉掉！** 单元测试是程序员的“防守阵地”。

13. 「リファクタリング (Refactoring / 重构)」

什么是 **リファクタリング (Refactoring)**？

- **核心定义**：它是一种“**只动内部，不动外部**”的优化行为。
- **本质**：在不改变软件功能（外部行为）的前提下，通过修改代码结构来提高可读性、可维护性。

核心考点——“**重构的边界**”：

- **红线**：一旦你修改了软件的功能，这就不是重构了（那是 Bug 修复或功能升级）。
- **目的**：为了降低技术债，让“将来的我”更容易修改代码。

在考试中，关于重构的判断题通常会设置以下逻辑陷阱：

- **陷阱 1：重构 = 修改功能？**
 - **✗ 错误**：这是最致命的陷阱。重构是“换个更优雅的方式实现相同的功能”，如果改了功能，那就是“功能变更”。
- **陷阱 2：重构 = 性能优化？**
 - **✗ 错误**：虽然重构有时能提升性能，但其首要目标是「**可読性 (可读性)**」和「**保守性 (可维护性)**」。不要为了性能而把代码写得晦涩难懂。

◦ **核心判断逻辑 (正确项特征):**

- 凡是提到「**振る舞いを変えずに (在不改变功能/行为的前提下)**」 + 「**内部構造を変える (改变内部结构)**」，这就是重构。

把「**リファクタリング**」脑补成【老王餐厅的‘后厨整容师’】：

 老王的‘调料瓶归位记’：

- 老王餐厅的调料瓶摆得乱七八糟。
- 老王把它们重新整理了一下，换成了透明盒子，贴上了标签（**重构**）。
- 注意：菜的味道（**软件的外部行为**）完全没变，客人吃不出区别。但是厨师找调料的速度快多了（**可读性/效率提升**）。

记住：重构 = “**给代码做整形，味道（功能）不能变，看起来更顺眼**”

 “重构”考场的“机械化”秒杀公式：

- a. **关键词锁定：**在选项中寻找「**外部から見たときの振る舞いを変えずに (不改变外部行为)**」。这是重构的唯一真理，看到这句话，基本就是正确答案。
- b. **绝对排雷 (见到直接 Pass)：**
 - 凡是说“添加新功能”、“修改功能逻辑”、“修复 Bug”的 → **通通划掉！** 这不属于重构。
 - 凡是提到“必须使用 CASE 工具”的 → **划掉！** 重构是开发者的日常代码清理，不需要特定的工具约束。

14. 「構成管理 (Configuration Management / 配置管理)」

什么是 構成管理 (Configuration Management)?

- **核心定义：**它是给软件开发中的所有“零件”（代码、文档、配置）建立档案，并严密监控它们“谁改了、改成啥样、版本对不对”的机制。

构成管理的“防线”：

- 它的作用是防止：**版本混乱、文档与代码不匹配、修改无法追溯、版本丢失/覆盖。**

核心考点——“边界思维”：

- 如果一个问题是关于“流程规范、版本一致性、履历追溯”的，那是构成管理的锅。
- 如果一个问题是关于“编程能力、工作量预估、测试质量”的，那是项目管理或质量管理的锅，与“构成管理”无关。

我们逐一拆解，找出那个“不是构成管理导致的”问题：

- **ア：開発者が定められた改版手続に従わずにプログラムを修正したため、今まで動作していたプログラムが、不正な動作をするようになった。**
 -  **构成管理的问题：**违反了“改版程序 (改版手続)”，即破坏了版本控制的规矩。属于构成管理不到位。

- イ：システムテストにおいて、単体テストレベルのバグが多発して、開発が予定どおり進捗しない。
 -  正确答案 (不是构成管理的问题)：这是「品質管理 (质量管理)」或「単体テストの不備 (单元测试缺失)」的问题。哪怕你把版本管理做得再好，如果代码质量本身烂、没测出来，它还是会出 Bug。这属于技术/质量范畴，不是“档案管理”的锅。
- ウ：仕様書、設計書及びプログラムの版数が対応付けられていないので、プログラム修正時にソースプログラムを解析しないと、修正すべきプログラムが特定できない。
 -  构成管理的问题：这是典型的“配置项不对应”问题。规格书、设计书、程序版本没对齐，导致找不到对应的零件，这是典型的配置识别与追踪失败。
- エ：一つのプログラムから多数の派生プログラムが作られているが、派生元のプログラム修正がすべての派生プログラムに反映されない。
 -  构成管理的问题：这是“分支管理 (Branch Management)”混乱，没做好派生源的同步与跟踪，属于配置管理不当。

把「构成管理不当的问题」脑补成【老王餐厅的‘版本乱斗’】：

 老王的‘乱七八糟档案柜’：

- ア (违反程序)：厨师没登记就乱改配方，锅里全是黑烟。 → 版本管不住。
- ウ (版数不对应)：设计书和代码不是一个版本的，厨师拿着去年的菜谱炒明年的菜。 → 档案不配套。
- エ (派生混乱)：改了主菜，分店的改动没跟上，味道全变了。 → 分支不同步。
- イ (Bug 多发)：菜不好吃、做菜慢、测试 Bug 多。 → 这纯粹是厨艺不行/测试没做好，和档案室里记没记录无关。

记住：构成管理是“管档案的”，Bug 质量是“管厨艺的”。不要把厨艺问题算在档案室头上！

**

口诀唤醒记忆：

构成管理管身份，版本记录要精准。

版数不符是管理，私自修改程序混。

Bug 多发测不准，技术质量是核心！

 “构成管理”考场的“机械化”秒杀公式：

a. 关键词诊断：

- 题目问“哪项不属于”，直接找“与版本、档案、履历、派生、匹配度无关”的选项。
- 看到“Bug 多发”、“进度慢”、“质量差”，这些通常是“测试过程”或“工程质量”的问题，而非“构成管理”的问题。

b. 绝对排雷：

- 不要被“系统测试”迷惑，虽然在测试阶段发生，但其根本原因如果是因为“版本不匹配”，那它就是构成管理问题；如果是因为“测试不充分”，那它就是质量管理问题。

15. 「スクラム (Scrum)」

Scrum 的核心逻辑：

- 将大型项目拆解为多个「**スプリント (Sprint / 冲刺)**」（通常为数周）。每个 Sprint 都是一个小型的“分析-设计-实现-测试”闭环，目的是为了**尽早交付可运行的功能**。

四大事件的职能分工：

- **スプリントプランニング (Sprint Planning)**：确定“做什么”和“怎么做”，是冲刺的**起点**。
- **デイリースクラム (Daily Scrum)**：每日 10-15 分钟的同步会，核心是“昨日工作、今日计划、障碍反馈”，目的是**促进沟通和调整**。
- **スプリントレビュー (Sprint Review)**：冲刺结束时的成果展示会，与利益相关者 (Stakeholder) 沟通并根据结果**决定后续方向**。
- **スプリントレトロスペクティブ (Sprint Retrospective)**：冲刺回顾会，复盘“哪些做得好、哪些没做好”，核心目的是**过程改善**。

我们可以通过辨析其核心职能来防范考场陷阱：

- **スプリントプランニング vs デイリースクラム**
 - 如果你看到题目问“确定目标”或“计划工作”，那一定是 Planning；如果题目描述“短时间”、“每日”、“障碍共享”，那一定是 Daily Scrum。
- **スプリントレビュー vs スプリントレトロスペクティブ**
 - 리뷰 (Review) 的焦点是「**プロダクト (产品)**」的成果（展示给别人看）；
 - レトロスペクティブ (Retrospective) 的焦点是「**チームプロセス (团队流程)**」的优化（自己关起门来反思）。

老王餐厅的‘每日流程’：

- **Planning (计划会)**：早上开会，老王大喊：“今天我们要搞定三道招牌菜，分工如下！”（**确定目标与计划**）。
- **Daily Scrum (每日站会)**：午饭前 10 分钟，厨师们站着围一圈：“我切好了肉，他炒好了菜，火不够大！”（**沟通进度、同步障碍**）。
- **Review (评审会)**：晚饭后，客人 (Stakeholder) 尝了新菜：“这菜不错，下次多加点辣。”（**展示成果、听反馈**）。
- **Retrospective (回顾会)**：关门后，厨师长把大家叫来：“今天这锅底粘锅了，明天我们换个不粘锅。”（**反思、流程改善**）

“Scrum 事件”考场的“机械化”秒杀公式：

a. 关键词诊断：

- 看到「起点/計画」→ 锁定 **プランニング**。
- 看到「毎日/10~15分/コミュニケーション」→ 锁定 **デイリースクラム**。
- 看到「成果物/プレゼン/ステークホルダー」→ 锁定 **レビュー**。
- 看到「振り返り/上手くいったこと/改善」→ 锁定 **レトロスペクティブ**。

b. 绝对避雷：

- 不要把“给利益相关者看成果 (Review)”和“团队内部改进流程 (Retrospective)”搞混，一个是**向外看产品**，一个是**向内看团队**。

16. 「スクラムの役割 (Scrum Roles)」

Scrum 团队中有三个关键角色，考试考点在于区分它们的“责任边界”：

- a. **プロダクトオーナー (PO)**：负责“做什么”和“价值最大化”，管菜单顺序。
- b. **開発チーム (Development Team)**：负责“怎么做”，管具体做菜和质量。
- c. **スクラムマスター (SM)**：负责“保证大家能顺利干活”，管扫清障碍和流程运作。

虽然当前主要是在讲「**プロダクトオーナー (PO)**」的定义：

- **PO 的核心逻辑**：管理「**プロダクトバックログ (Product Backlog)**」。
- **为什么这是 PO 的职责？** 因为 PO 是对“产品方向”和“投资回报”负责的人。如果由开发团队管バックログ，他们可能会偏向技术实现而忽略商业价值；如果由 SM 管，那就越权了。
- **其他角色的边界**：
 - 如果题目问“交付高质量成果”，那是 **開発チーム** 的责任。
 - 如果题目问“消除障碍”、“促进团队运作”，那是 **スクラムマスター** 的责任。

【Scrum 厨房分工图】

- **PO = “金牌经理”**：只管“顾客想吃啥”和“卖什么最赚钱”（管 Backlog，定价值）。
- **开发团队 = “大厨们”**：只管“菜怎么做才好吃，才符合卫生标准”（管实现，管质量）。
- **SM = “后勤大管家”**：只管“厨师心情好不好、刀具磨没磨、流程顺不顺”（管流程，扫障碍）。

“Scrum 角色”秒杀公式：

- a. 看到“价值”、“优先级”、“Backlog”→ 无脑锁定 **PO**。
- b. 看到“实现”、“高质量成果”、“设计/测试”→ 无脑锁定 **开发团队**。
- c. 看到“支援”、“障碍排除”、“促进”→ 无脑锁定 **SM**。

17. 「スパイラルモデル (Spiral Model / 螺旋模型)」

スパイラルモデル (Spiral Model) 的核心定位：

- 它是“**风险驱动 (Risk-Driven)**”的模型。它的名字来源于“螺旋”，意味着过程不是一次性完成，而是像旋转上升的楼梯一样，一层一层地优化。

核心逻辑：

- **分割与循环**：把大系统拆成小模块，每个模块循环执行：「計画 → リスク分析 → 設計・開発 → 評価」。
- **螺旋本质**：每一次循环（一圈）都比上一圈更接近成品，且在每一圈开始前都先做「リスク分析 (风险分析)」。

在 FE 考试中，关于螺旋模型，你只需要记住它的三个关键标签：

- **标签 1：リスク分析 (风险分析)**
 - 这是它的“看家本领”。如果选项里提到“在开发过程中识别并降低风险”，那基本就是它。
- **标签 2：反復 (迭代)**
 - 它不是“瀑布 (Waterfall)”那样一条路走到黑。它通过重复循环来逐步完善系统，非常适合需求不明确、技术难度大的项目。
- **陷阱辨析：**
 - 不要将其与「ウォーターフォール (瀑布模型)」混淆（瀑布是一次性完成）。
 - 不要将其与「プロトタイプ (原型模型)」混淆（原型模型侧重于“确认需求”，而螺旋模型侧重于“全程风险管理”）。

老王的‘螺旋攀登法’：

- 老王要攀登“系统开发”这座高峰，他不打算一步冲顶（那是瀑布，容易摔死）。
- 他走螺旋楼梯，每爬一圈，先拿望远镜看看前方有没有悬崖（风险分析）。
- 确定没悬崖了，再往上爬一圈（开发），然后站在平台上评估一下刚才爬得对不对（评价）。
- 这样绕着圈爬，虽然慢，但绝对不会掉下山谷！

记住：螺旋模型 = “爬山要绕圈，环环有评估，步步避风险，稳稳到顶峰”

“螺旋模型”考场的“机械化”秒杀公式：

- 关键词锁定**：在选项中寻找「リスク (风险)」、「繰り返 (重复/迭代)」、「段階的 (阶段性)」。
- 绝对排雷 (见到直接 Pass)：**
 - 如果选项说“一次性完成所有功能” → **叉掉！** (这是瀑布)。
 - 如果选项说“完全不需要风险分析，只看客户反馈” → **叉掉！** (这是原型或敏捷)。

18. 「バーンダウンチャート (Burndown Chart)」

バーンダウンチャート (Burndown Chart) 的核心逻辑：

- **“Burn” (燃尽) + “Down” (下降)**：这是它的灵魂。它是一张通过“消耗剩余工作量”来直观展示进度是否“按计划归零”的图表。

核心三要素：

- **纵轴 (Y轴)：** 残作业量 (Remaining Work) —— 必须要“减小”。
- **横轴 (X轴)：** 时间 (Time/Days) —— 随着项目推进而“延伸”。
- **目的：** 确保剩余的工作量随着时间推移不断“燃烧殆尽”，直到变成 0。

縦軸を残作業の量，横軸を時間とし，プロジェクトが進むに従って残作業の量が減少する様子を確認する。

-  **正确答案：** 完美命中！纵轴是“残作业量”，随着横轴“时间”推移而“减少”，这就是标准的燃尽定义。

把「燃尽图」脑补成【老王餐厅的‘食材消灭表’】：



老王的‘剩余消灭法’：

- 这图就像一根**“倒计时蜡烛”**。
- 开始时蜡烛最长（剩余作业量最大），随着时间推移，蜡烛越烧越短（残作业量减少）。
- 直到最后烧完了（残作业量为 0），项目就圆满结束了。

记住：燃尽图，就是“看蜡烛剩多少”。线越低，离胜利越近！



“燃尽图”考场的“机械化”秒杀公式：

a. 词汇对对碰：

- 看到 **バーンダウン (Burndown)** → 必须选「残作业量 (Remaining Work)」 + 「减少 (減少)」。
- 看到 **バーンアップ (Burnup)** → 必须选「完成量 (Finished Work)」 + 「增加 (増加)」。

b. 绝对避雷：

- 凡是选项里说“工作量增加”或“增减”的，直接叉掉，那不是“燃尽”该干的事。

c. 最终逻辑：燃尽图存在的唯一目的，就是为了让别人一眼看出“还要做多少活儿”。

19. 「組込みシステム (嵌入式系统) 开发环境的维护」

开发环境维护的核心原则：

- 嵌入式产品的生命周期极长（可能长达 10 年甚至更久）。
- **核心逻辑：** 只要产品还在维修期或需要后续发布补丁 (Patch/Update)，该产品的「開発環境 (开发环境)」就必须作为核心资产被**长期、稳定地保存**，以确保任何时候都能重新编译出相同的固件 (Firmware)。

核心考点——“一致性”：

- 必须保证“编译环境”的一致性。如果更新了环境（如升级编译器），可能导致编译出的二进制文件与原版不符，从而产生不可预见的 Bug。

開発環境は、使用頻度に関係なく、定期的に動作確認などを行って維持管理すべきである。

-  **正确答案**：完美命中！为了保证在需要的时候（比如 5 年后）环境还能用，必须定期检查它的完整性，防止物理损坏、磁盘故障或系统过期，确保其处于“**随时待命**”状态。

把「**嵌入式开发环境维护**」脑补成【老王餐厅的‘**时光胶囊冷库**’】：

 老王的‘**时光胶囊**’：

- 老王做了一道绝世“招牌红烧肉”，用的是特定老灶（**嵌入式开发环境**）。
- 如果 5 年后客人嫌味道变了，老王必须用**当年的老灶**复刻出来。
- 因此，哪怕这灶平时不用（**不常使用**），老王也得定期擦擦灰，检查有没有生锈（**定期维护/动作确认**）。
- *切记：千万不能换成新烤箱，味道会变的！*

记住：环境是时光机，千万别乱升级。定期维护擦灰土，随时开机能应敌。

 “开发环境维护”考场的“机械化”秒杀公式：

- a. **关键词锁定**：看到“嵌入式开发环境”，答案必须选「**维持现状、定期检查、长期保存**」。
- b. **绝对排雷** (见到直接 Pass)：
 - 凡是建议“升级/更新”的 → **通通划掉！** (升级=毁灭)。
 - 凡是说“不用留、扔掉”的 → **划掉！** (这是不负责任的开发)。
 - 凡是推给“第三方责任”的 → **划掉！** (这是甩锅思维)。

20. 「マッシュアップ (Mashup / 混搭)」

核心本质：Mashup 是「**API 驱动的高式开发**」。它的核心逻辑不是“重写”，而是“组合”已有的现成功能。

三大类型（考点高频区）：

- **呈现层 (Presentation)**：侧重“看”，只是把别人的网页窗体嵌进自己的页面。
- **数据层 (Data)**：侧重“找”，把分散在各处的数据抓回来，加工成一张表或图。
- **逻辑层 (Logic)**：侧重“做”，把 A 的输出作为 B 的输入，形成一套完整的业务链。

在 FE 考试中，Mashup 的题目通常会考这三种类型的区别，我们需要用“透视镜”来看它们：

- **プレゼンテーションマッシュアップ (呈现层)**：
 - **深度解析**：最浅层。就像“引用链接”或“嵌入框架 (iframe)”。
 - **关键词**：表示 (表示)、コンテンツ (内容)、Webページ (网页)。
- **データマッシュアップ (数据层)**：
 - **深度解析**：中层。把乱七八糟的源数据（如两家公司的股价数据）整合在一起。
 - **关键词**：統合 (整合)、可視化 (可视化)、データ (数据)。
- **ロジックマッシュアップ (逻辑层)**：

- **深度解析**：最深层。处理的是“**工作流 (Workflow)**”，即处理逻辑的串联。
- **关键词**：ワークフロー (工作流)、入出力 (输入输出)、連結 (连接)。

把「**Mashup 三兄弟**」脑补成【**老王餐厅的‘超级拼盘流水线’**】：

 **老王的‘拼盘三境界’：**

- **呈现层 (展示)**：老王在门口摆个电视机，循环播放隔壁店的菜品视频。客人在老王店里看视频。(展示内容)
- **数据层 (整合)**：老王拿个本子，把隔壁店的价格和自己店的价格写一起，做成一张对比表。(数据整合)
- **逻辑层 (流程)**：老王和隔壁店打通了，客人先在老王这交钱，系统自动给隔壁店下单做菜，最后端回老王店。(流程串联)

 **“Mashup 类型”考场的“机械化”秒杀公式：**

a. **关键词扫描：**

- 看到“**网页内表示 (Webページ内表示)**” → 绝对是 **プレゼンテーション**。
- 看到“**数据的收集、整合、统计**” → 绝对是 **データ**。
- 看到“**流程 (Workflow)、调用、入出力连结**” → 绝对是 **ロジック**。

b. **绝对避雷：**

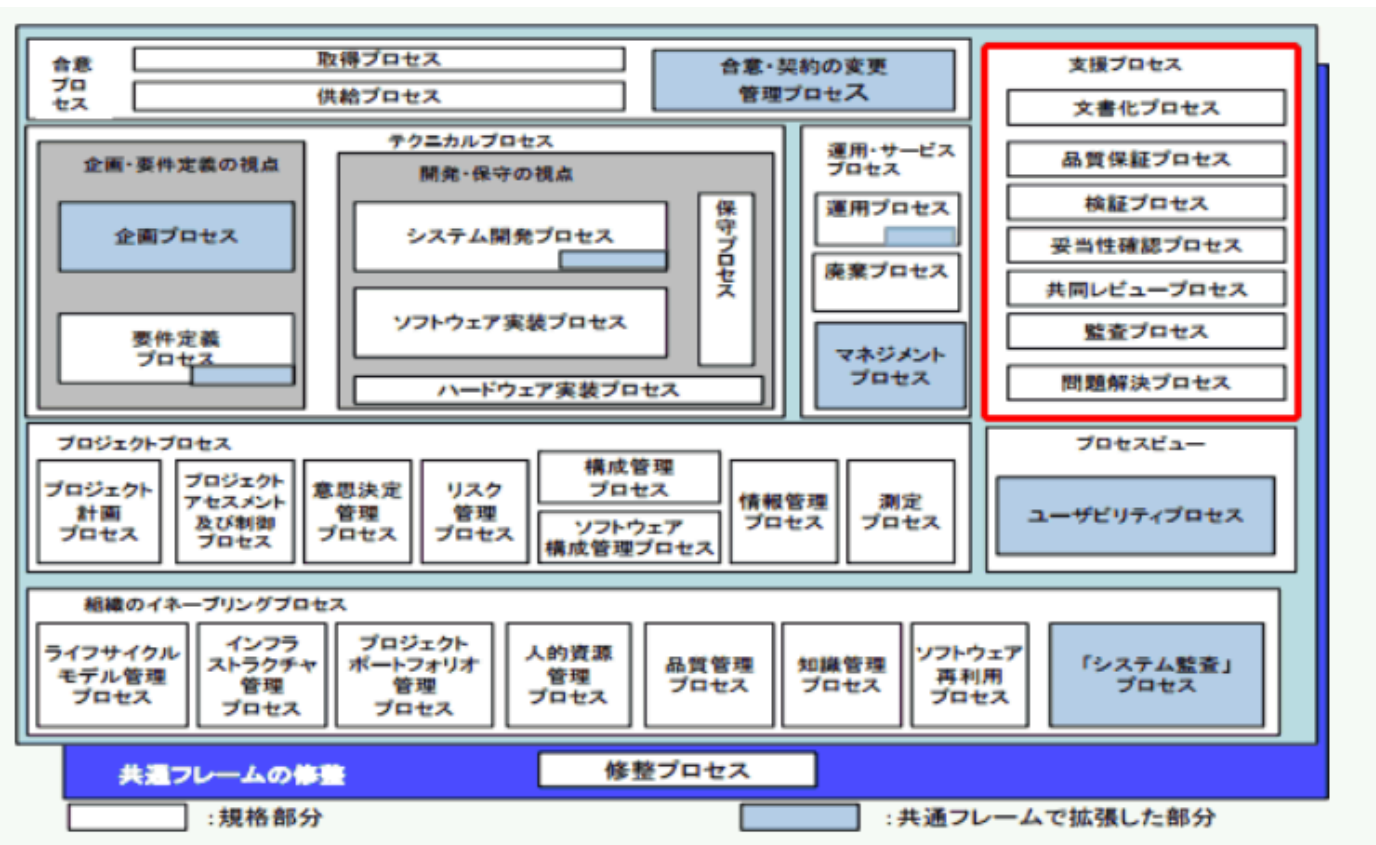
- 别被“**集成**”这两个字带偏，只有涉及到“**Workflow/输入输出连接**”的才是逻辑混搭。

21. 「**共通フレーム (Common Framework/SLCP)**」这一在 FE 考试中极易混淆的核心考点：「**検証 (Verification)**」与「**妥当性確認 (Validation)**」。

区分“検証”与“妥当性確認”的核心逻辑：

- **検証 (Verification)**：问的是“我们做出来的东西符合设计规范吗？”（关注的是：产品是否满足之前的要求/设计图）。
- **妥当性確認 (Validation)**：问的是“我们做出来的东西真的是用户想要的吗？”（关注的是：产品是否满足用户的实际目的/意图）。

本题考点：题目问的是“是否符合利用者的意图 (意図された正しいもの)”，这是 Validation 的天职。



把「Verification vs Validation」脑补成【老王餐厅的‘出菜检查员’】：

🔍 老王的‘两道检查关’：

- 検証 (验证) = “对图纸”：检查员拿着配方表看，盐放了 5 克吗？锅热够 3 分钟吗？（符合规格要求）。
- 妥当性確認 (确认) = “问客人”：把菜端给客人，客人尝了一口说：“这正是我想吃的味道！”（符合用户意图）。

记住：验证是**对设计图**，确认是**对用户心**。题目一提到“用户意图”，就是“妥当性确认”！

🎯 “验证与确认”考场的“机械化”秒杀公式：

a. 关键词扫描：

- 看到「規定/仕様/設計 (规范/规格/设计)」→ 选 検証 (Verification)。
- 看到「利用者の視点/意図/要求 (用户视角/意图/需求)」→ 选 妥当性確認 (Validation)。

b. 视觉识别法：

- Validation (妥当性确认) 的英文 V 开头，你可以想象它是“站在用户那边”的 V (Victory)。只要是满足用户最终意图的，就是它！

22. 「アジャイル開発 (Agile Development)」 中的核心概念「イテレーション (Iteration / 迭代)」

什么是“イテレーション (Iteration)”？

- **核心定义**：它不是一次性把项目做完，而是将开发周期切分为多个“数周左右的短周期”。
- **核心动作**：在每个周期内，完整经历“分析 -> 设计 -> 实现 -> 测试”的闭环，产出一个可运行的、可展示的软件功能片段。

老王的‘周周试吃会’：

- 以前老王一次性做满汉全席，做完客人说不好吃，全毁了（瀑布模型）。
- 现在老王搞“迭代”：这周只做一道鱼，端给客人尝（交付/反馈）。
- 客人说：“不够辣！”
- 下周老王马上改进，做出更辣的鱼，并加上新的汤（响应变化）。

记住：迭代 = “切分短周期，快速给试吃，客人说不准，马上改下次”

“迭代 (Iteration)” 考场的“机械化”秒杀公式：

- 关键词锁定：看到「イテレーション」，立刻寻找「顧客の要求 (客户需求)」、「不一致の解消 (解决不一致)」、「変化への対応 (应对变化)」、「短いサイクル (短周期)」。
- 绝对排雷 (见到直接 Pass)：
 - 凡是把“迭代”定义为“晨会”、“看板”、“结对编程”等具体动作的选项 → 通通划掉！那些是迭代中的“手段”，不是迭代本身的“目的”。
- 最终思维：迭代是为了“降低试错成本”。只要题目描述的是为了“适应变化、反馈改进”的，那它就是迭代。

23. 「プロトタイプ (Prototype / 原型模型)」

题目：プロトタイプを1回作成するごとに未確定な仕様の50%が確定するとき、プロトタイプ開始時点で未確定だった仕様の90%以上を確定させるには、プロトタイプを何回作成する必要があるか。

原型模型的核心逻辑：

- 这是一个「剩余量持续递减」的累加模型。
- 关键公式：下一次的累计确定率 = 上一次的累计确定率 + (剩余未确定率 × 50%)。

核心考点——“确定率”的陷阱：

- 不要只看“每次解决了多少”，要看“总共累计解决了多少”。

我们用“进度仪表盘”来一步步推演，直到它跨越 90% 的门槛：

- 第 1 次：解决了 100% 的 50% → 确定率 50%。
- 第 2 次：在剩下的 50% 中解决一半（25%） → 累计确定 50% + 25% = 75%。
- 第 3 次：在剩下的 25% 中解决一半（12.5%） → 累计确定 75% + 12.5% = 87.5%（注：还没到 90%，必须继续）。
- 第 4 次：在剩下的 12.5% 中解决一半（6.25%） → 累计确定 87.5% + 6.25% = 93.75%（达成 90% 以上！）。
- 结论：达成目标需要 4 次。如果原题选项中“工”对应的是“4”，那么“工”就是标准答案。

老王的‘五折消除术’：

- 你有 100% 的难题。
- 每做一次原型，我就帮你砍掉剩下难题的一半（五折优惠）。
- 剩下的难题变成了：50% → 25% → 12.5% → **6.25%**。
- 看！只要剩下小于 10%，你就赢了！

记住：别管加法，盯着“剩余未确定部分”看。它是 50% → 25% → 12.5% → 6.25%。当你看到 6.25% 时，它已经是第 4 次了！

 “原型模型计算”考场的“机械化”秒杀公式：

a. 快速列表法：考试时在草稿纸上直接列出“剩余百分比”：

- 1回: 50%
- 2回: 25%
- 3回: 12.5%
- 4回: 6.25%

b. 判定界限：

- 看到题目问“90% 以上确定” $\rightarrow$ 等同于问“剩余不到 10%”。
- 对比列表，6.25% < 10%，所以选 4 次。

c. 防坑指南：

- 千万不要简单地把 50% 乘以次数，那是线性增长，原型模型是“**边际递减**”的（越往后越难确定）。